

Appunti di Calcolatori - Introduzione

Trascrizione dalle fonti del corso

Indice

1	Introduzione ai Calcolatori	4
1.1	Descrizione del Corso	4
1.2	Struttura e Modalità d'Esame	4
1.3	Evoluzione Storica: dall'ENIAC ad oggi	4
1.4	Perché studiare l'architettura?	4
1.5	Software di Sistema e Linguaggi	4
1.6	Componenti del Calcolatore	5
2	Aritmetica dei Calcolatori	5
2.1	La Rappresentazione Digitale	5
2.2	Sistemi di Numerazione Posizionali	5
2.3	Conversioni di Base	5
2.4	Operazioni Aritmetiche e Shifting	6
2.5	Rappresentazione dei Numeri Interi	6
2.6	Overflow	6
3	Rappresentazione dei Numeri Reali	6
3.1	Limiti della Rappresentazione	6
3.2	Rappresentazione in Virgola Fissa (Fixed Point)	6
3.3	Virgola Mobile (Standard IEEE 754)	7
3.4	Formati e Precisione	7
3.5	Valori Speciali	7
4	Codifica del Testo	7
4.1	Dal Carattere al Bit	7
4.2	Lo Standard ASCII	8
4.3	Unicode e UTF-8	8
4.4	Esempio di Codifica	8
5	Cenni alle Reti Logiche	8
5.1	Valori Logici ed Elettronica	8
5.2	Tipologie di Reti Logiche	9
5.3	Algebra di Boole e Porte Logiche	9
5.4	Circuiti Combinatori Fondamentali	9
5.5	Elementi di Memoria	9

6	Il Linguaggio Assembly	10
6.1	Il Gap tra Umano e Calcolatore	10
6.2	Instruction Set Architecture (ISA)	10
6.3	Operandi e Registri	10
6.4	Controllo del Flusso e Salti	10
6.5	Principi di Progettazione dell'ISA	11
7	L'Architettura RISC-V	11
7.1	Caratteristiche Generali	11
7.2	I Registri nel RISC-V	11
7.3	Organizzazione della Memoria	11
7.4	Trasferimento Dati: Load e Store	12
7.5	Rappresentazione delle Istruzioni	12
7.6	Pseudo-istruzioni	12
8	Tutorato Assembly RISC-V	12
8.1	Concetti Teorici Fondamentali	12
8.2	Traduzione di Costrutti: dal C all'Assembly	13
9	Soluzioni Mockup Complete	14
10	Flusso di Controllo e Procedure	24
10.1	Istruzioni di Salto Condizionato	24
10.2	Protocollo di Chiamata delle Procedure	24
10.3	Gestione dello Stack e Record di Attivazione	25
10.4	Funzioni Foglia e Non Foglia	25
10.5	Organizzazione della Memoria	25
11	L'Architettura Intel x86	25
11.1	Evoluzione e Caratteristiche CISC	25
11.2	I Registri Intel	26
11.3	Formato delle Istruzioni e Operandi	26
11.4	Modalità di Indirizzamento	26
11.5	Istruzioni Particolari	26
11.6	Convenzione di Chiamata (ABI)	27
12	L'Architettura Intel x86	27
12.1	Evoluzione e Caratteristiche CISC	27
12.2	I Registri Intel	27
12.3	Formato delle Istruzioni e Operandi	27
12.4	Modalità di Indirizzamento	28
12.5	Istruzioni Particolari	28
12.6	Convenzione di Chiamata (ABI)	28
13	L'Architettura ARM	28
13.1	Origini e Caratteristiche	28
13.2	I Registri ARM	28
13.3	Formato delle Istruzioni e Flessibilità	29
13.4	Esecuzione Condizionale	29

13.5	Modalità di Indirizzamento e Write-back	29
13.6	Istruzioni Multiple (LDM e STM)	29
14	La Toolchain: dalla Sorgente all'Esecuibile	30
14.1	Il Processo di Traduzione	30
14.2	Il Ruolo dell'Assembler	30
14.3	Struttura del File Oggetto	30
14.4	Il Linker e la Creazione dell'Esecuibile	31
14.5	Linking Statico e Dinamico	31

1 Introduzione ai Calcolatori

1.1 Descrizione del Corso

Il corso, noto anche come **Architettura degli elaboratori**, si concentra principalmente su lezioni teoriche integrate da esercitazioni pratiche riguardanti l'aritmetica dei calcolatori e il linguaggio **Assembly**. In totale, l'insegnamento prevede **48 ore** di didattica frontale distribuite in 12 settimane.

1.2 Struttura e Modalità d'Esame

Il programma è suddiviso in tre parti principali:

1. **Prima parte:** Introduzione, aritmetica dei calcolatori e cenni sulle reti logiche.
2. **Seconda parte:** Linguaggio Assembly, con focus su architetture RISC-V, INTEL e ARM.
3. **Terza parte:** Elementi di architettura dei calcolatori, inclusi CPU, Pipeline e gerarchie di memoria.

L'esame prevede una **prova scritta**, spesso svolta su piattaforma Moodle con quesiti a risposta multipla, e un **orale opzionale**. Quest'ultimo è obbligatorio per voti compresi tra 15 e 17 o per confermare voti alti come la lode. Il libro di riferimento è *“Struttura e progetto dei calcolatori - Progettare con RISC-V”* di Patterson e Hennessy.

1.3 Evoluzione Storica: dall'ENIAC ad oggi

I calcolatori elettronici rappresentano oggi una tecnologia vitale che produce il 10% del PIL degli Stati Uniti.

- **ENIAC (1946):** Considerato uno dei primi calcolatori, fu commissionato per calcolare le traiettorie dei proiettili; occupava una stanza di 9x30 metri e consumava enormi quantità di energia.
- **La rivoluzione:** Attualmente i calcolatori pervadono ogni aspetto della vita, dalle automobili ai telefoni cellulari, fino alla robotica e alla mappatura del genoma.

1.4 Perché studiare l'architettura?

È essenziale per un programmatore conoscere l'organizzazione del calcolatore per scrivere software efficiente, comprendendo come sfruttare la **gerarchia di memoria** e il **parallelismo**. Le prestazioni dipendono dall'interazione tra algoritmi, compilatore, hardware e l'**ISA** (Instruction Set Architecture), che definisce le istruzioni riconosciute dalla CPU.

1.5 Software di Sistema e Linguaggi

Il software di sistema agisce come intermediario tra l'utente e l'hardware:

- **Sistema Operativo (SO):** Gestisce l'input/output, la memoria e il multitasking.
- **Compilatore:** Traduce il codice ad alto livello (come il C) in linguaggio macchina.

L'**Assembly** introduce codici mnemonici per rendere le istruzioni macchina gestibili dall'uomo prima della traduzione finale in bit.

1.6 Componenti del Calcolatore

Un calcolatore è composto da quattro elementi principali:

1. **Input:** Dispositivi per inviare dati, come tastiera e mouse.
2. **Output:** Dispositivi per visualizzare i risultati, come schermi e stampanti.
3. **Processore:** Esegue le elaborazioni, diviso in **datapath** (parte operativa) e **unità di controllo**.
4. **Memoria:** Unità dove vengono memorizzati dati e programmi in esecuzione.

2 Aritmetica dei Calcolatori

2.1 La Rappresentazione Digitale

Un computer è costituito da circuiti elettronici in cui l'informazione è manipolata esclusivamente come sequenze di bit, dove il valore **1** indica il passaggio di corrente e lo **0** l'assenza.

- **Bit:** L'unità minima di informazione (0 o 1).
- **Byte:** Una sequenza raggruppata di 8 bit.
- **Codifica:** Funzione che associa un simbolo a una sequenza di bit. Una codifica su n bit permette di rappresentare 2^n simboli distinti.

2.2 Sistemi di Numerazione Posizionali

I calcolatori utilizzano sistemi posizionali in cui il valore di una cifra dipende dalla sua posizione. Il valore di una sequenza di cifre in base B è dato dalla formula:

$$\sum_{k=0}^i c_k \cdot B^k$$

dove c_k è la cifra in posizione k . Le basi principali sono:

- **Base 10 (Decimale):** Utilizza le cifre da 0 a 9.
- **Base 2 (Binaria):** Base nativa dei circuiti, utilizza solo 0 e 1.
- **Base 16 (Esadecimale):** Utilizza 16 cifre (0-9 e A-F) per rappresentare in modo compatto lunghe sequenze binarie (un byte corrisponde a due cifre esadecimali).

2.3 Conversioni di Base

- **Da Binario a Decimale:** Si sommano le potenze di 2 corrispondenti ai bit impostati a 1.
- **Da Decimale a Binario:** Si utilizza il metodo delle **divisioni successive** per 2, dove i resti (letti dall'ultimo al primo) formano il numero binario.

2.4 Operazioni Aritmetiche e Shifting

Le operazioni di somma e sottrazione binaria seguono regole simili al sistema decimale, gestendo riporti e prestiti. L'operazione di **shifting** consiste nello spostare i bit: aggiungere uno 0 a destra equivale a moltiplicare per 2, mentre traslare i bit a destra eliminando l'ultima cifra equivale a dividere per 2. I compilatori usano lo shift per ottimizzare i calcoli.

2.5 Rappresentazione dei Numeri Interi

Per gestire i numeri negativi si utilizzano diverse tecniche:

- **Modulo e Segno:** Il bit più significativo indica il segno (0 per +, 1 per -). Presenta l'inconveniente della doppia rappresentazione dello zero.
- **Complemento a 1 (CA1):** I numeri negativi si ottengono invertendo tutti i bit del positivo. Anche qui lo zero ha due codifiche.
- **Complemento a 2 (CA2):** È lo standard moderno. Un numero negativo si ottiene facendo il complemento a 1 e sommando 1.
 - **Vantaggi:** Codifica dello zero unica e sottrazioni eseguibili come somme ($x - y = x + \text{comp2}(y)$).
 - **Intervallo:** Con k bit si rappresentano numeri da -2^{k-1} a $2^{k-1} - 1$.

2.6 Overflow

L'overflow si verifica quando il risultato di un'operazione non è rappresentabile con i bit a disposizione. In aritmetica CA2, l'overflow può avvenire solo se si sommano due numeri con lo **stesso segno** e il risultato presenta il segno opposto.

3 Rappresentazione dei Numeri Reali

3.1 Limiti della Rappresentazione

Rappresentare esattamente un numero reale $x \in \mathbb{R}$ non è sempre possibile all'interno di un calcolatore. Questo limite riguarda in particolare i numeri irrazionali (come π) o quelli con una sequenza infinita di cifre decimali non periodiche, che non possono essere contenuti in un numero finito di bit. Per tali ragioni, i calcolatori operano spesso con approssimazioni dei valori reali.

3.2 Rappresentazione in Virgola Fissa (Fixed Point)

In questa codifica, su un totale di k cifre, una parte fissa f viene dedicata alla parte frazionaria e la restante parte $k - f$ alla parte intera.

- **Vantaggi:** La semplicità delle operazioni aritmetiche, che vengono eseguite come se fossero numeri interi per poi essere riscalate, semplificando il progetto della CPU. Se il numero è rappresentabile, non vengono introdotti errori di approssimazione.
- **Svantaggi:** Diventa inefficiente quando l'applicazione deve gestire contemporaneamente numeri molto grandi e numeri molto piccoli (ordini di grandezza diversi).

3.3 Virgola Mobile (Standard IEEE 754)

La **Virgola Mobile** (Floating Point) risolve i limiti della virgola fissa esprimendo il numero nella forma $x = M \cdot B^E$, dove M è la mantissa ed E l'esponente. Lo standard universale è l'**IEEE 754**, che definisce tre componenti principali:

1. **Segno (s)**: 1 bit (0 per positivo, 1 per negativo).
2. **Esponente (E)**: Memorizzato in notazione "eccesso" (bias) per permettere la gestione di esponenti negativi.
3. **Mantissa (M)**: Rappresenta la parte frazionaria; nei numeri normalizzati si assume la presenza di un "1" implicito a sinistra della virgola.

3.4 Formati e Precisione

I calcolatori utilizzano principalmente due formati:

- **Precisione Singola (float)**: 32 bit totali (1 segno, 8 esponente, 23 mantissa). Il bias è 127.
- **Precisione Doppia (double)**: 64 bit totali (1 segno, 11 esponente, 52 mantissa). Il bias è 1023.

3.5 Valori Speciali

Lo standard IEEE 754 definisce configurazioni specifiche per gestire eccezioni o limiti hardware:

- **Zero**: Esponente 0 e mantissa 0. Esiste sia lo zero positivo che quello negativo.
- **Infinito ($\pm\infty$)**: Esponente massimo (255 in precisione singola) e mantissa 0.
- **NaN (Not a Number)**: Esponente massimo e mantissa diversa da zero; indica operazioni non valide come $0/0$.
- **Numeri Denormalizzati**: Usati per gestire l'underflow quando un numero è troppo piccolo per essere normalizzato. Hanno esponente 0 e mantissa non nulla.

4 Codifica del Testo

4.1 Dal Carattere al Bit

Il testo è gestito dai calcolatori come una sequenza di caratteri, ciascuno dei quali deve essere mappato in una sequenza di bit attraverso una funzione di codifica. Una codifica che utilizza n bit permette di rappresentare un totale di 2^n simboli distinti.

4.2 Lo Standard ASCII

L'**ASCII** (*American Standard Code for Information Interchange*) è lo standard storico per la codifica dell'alfabeto anglosassone.

- **ASCII Standard:** Utilizza **7 bit** per rappresentare 128 caratteri, inclusi numeri, lettere maiuscole e minuscole, punteggiatura e caratteri di controllo.
- **ASCII Esteso:** Poiché i calcolatori lavorano su byte (8 bit), l'ottavo bit è stato utilizzato per raddoppiare i simboli disponibili (256 in totale). Tuttavia, non esiste un unico standard esteso; diverse tabelle (come la **ISO 8859-1** per l'Europa Occidentale) associano simboli diversi (es. lettere accentate) ai valori superiori a 127, creando problemi di compatibilità tra sistemi diversi.

4.3 Unicode e UTF-8

Per superare i limiti dell'ASCII e rappresentare tutti gli alfabeti mondiali (come il cirillico, il cinese o l'arabo), è stato introdotto lo standard **Unicode**.

- **Unicode:** Può codificare potenzialmente fino a 2^{32} simboli diversi.
- **UTF-8:** È la codifica più diffusa per Unicode. Utilizza un numero variabile di byte (da 1 a 4) per ogni carattere ed è **retrocompatibile con l'ASCII**, poiché i primi 128 caratteri Unicode corrispondono a quelli ASCII e occupano un solo byte.

4.4 Esempio di Codifica

Prendendo come esempio la parola “Ciao”, la sua rappresentazione in ASCII/UTF-8 segue la mappatura dei singoli caratteri in valori decimali, poi convertiti in bit:

- **C:** 67 ($0x43$) $\rightarrow$ 01000011
- **i:** 105 ($0x69$) $\rightarrow$ 01101001
- **a:** 97 ($0x61$) $\rightarrow$ 01100001
- **o:** 111 ($0x6F$) $\rightarrow$ 01101111

5 Cenni alle Reti Logiche

5.1 Valori Logici ed Elettronica

I calcolatori moderni sono realizzati mediante circuiti elettronici digitali che operano su due livelli di tensione fondamentali.

- **Alto, asserito (1):** Associato alla tensione di alimentazione V_{dd} .
- **Basso, negato (0):** Associato alla massa (tensione nulla).

Eventuali livelli di tensione intermedi sono considerati non significativi e assunti solo durante le fasi transitorie.

5.2 Tipologie di Reti Logiche

Le reti logiche sono circuiti che trasformano valori logici in ingresso in valori logici in uscita e si dividono in due categorie principali:

- **Reti Combinatorie:** L'uscita in ogni istante dipende esclusivamente dai valori degli ingressi in quell'istante. Non possiedono memoria del passato.
- **Reti Sequenziali:** L'uscita dipende sia dagli ingressi attuali che dalla storia degli ingressi passati. Queste reti possiedono una memoria interna, definita **stato** della rete.

5.3 Algebra di Boole e Porte Logiche

Le funzioni logiche possono essere espresse in modo compatto tramite l'algebra di Boole, basata su tre operatori fondamentali implementati fisicamente dalle corrispondenti **porte logiche**:

1. **AND** ($\cdot$): Produce 1 solo se tutti gli operandi sono 1.
2. **OR** ($+$): Produce 1 se almeno uno degli operandi è 1.
3. **NOT** ($\bar{A}$): Inverte il valore logico dell'ingresso.

Sono di fondamentale importanza le **leggi di De Morgan**, che permettono di ricavare qualsiasi operatore logico partendo da porte di tipo NAND o NOR.

5.4 Circuiti Combinatori Fondamentali

Tra i blocchi logici più utilizzati nell'architettura di un processore si distinguono:

- **Decoder:** Un circuito con n ingressi e 2^n uscite; in base alla combinazione binaria in ingresso, viene attivata una sola delle uscite.
- **Multiplexer (Mux):** Un selettore che, in base a un segnale di controllo, determina quale tra diversi ingressi dati deve essere trasferito in uscita.

5.5 Elementi di Memoria

La capacità di memorizzare informazioni in una rete logica si ottiene attraverso la **retroazione** (o reazione), ovvero riportando l'uscita di una porta in ingresso ad altre porte dello stesso stadio.

- **Latch S-R:** Elemento base con ingressi *Set* e *Reset*. Presenta lo svantaggio di uno stato "indecidibile" quando entrambi gli ingressi sono a 1.
- **Latch D (Gated):** Risolve l'ambiguità utilizzando un unico ingresso dati e un segnale di abilitazione chiamato **Clock**.
- **Flip-Flop:** Circuiti **edge-triggered** che caricano l'input solo sul fronte (salita o discesa) del clock, garantendo stabilità e prevenendo situazioni di corsa critica (*race condition*).
- **Registri:** Collezioni di flip-flop utilizzati per memorizzare parole di dati (*word*) all'interno del processore.

6 Il Linguaggio Assembly

6.1 Il Gap tra Umano e Calcolatore

I calcolatori elettronici sono in grado di comprendere ed eseguire esclusivamente il proprio **linguaggio macchina**, ovvero una sequenza di cifre binarie (0 e 1). Al contrario, i programmatori utilizzano solitamente linguaggi ad **alto livello** (come C, C++, Java) che sono astratti e comprensibili dall'uomo, ma non direttamente dalla CPU.

L'**Assembly** rappresenta il punto di incontro: utilizza **codici mnemonici** (come `add`, `sub`, `lw`) al posto delle sequenze di bit, mantenendo una stretta corrispondenza con le istruzioni macchina. Un programma chiamato **Assembler** ha il compito di tradurre questi codici mnemonici nella sequenza finale di bit.

6.2 Instruction Set Architecture (ISA)

L'**Instruction Set** è il vocabolario di istruzioni che una CPU è in grado di riconoscere. Sebbene ogni processore abbia il proprio, le differenze tra di essi sono spesso paragonabili a inflessioni regionali di una stessa radice linguistica. Le scelte di progettazione di un ISA si basano su criteri pratici: semplicità dei dispositivi, chiarezza delle applicazioni e velocità di esecuzione.

Le architetture si dividono principalmente in due categorie:

- **RISC (Reduced Instruction Set Computer)**: Come il **MIPS** o il **RISC-V**, caratterizzate da istruzioni semplici, di lunghezza fissa e un numero elevato di registri. L'accesso alla memoria avviene solo tramite istruzioni specifiche di *load* e *store*.
- **CISC (Complex Instruction Set Computer)**: Come l'architettura **Intel x86**, con istruzioni di lunghezza variabile e operazioni che possono accedere direttamente alla memoria.
- **ARM**: Un'architettura "pragmatica" che mescola caratteristiche RISC con modalità di indirizzamento più complesse.

6.3 Operandi e Registri

A differenza dei linguaggi ad alto livello, dove le variabili sono potenzialmente infinite, in Assembly gli operandi delle istruzioni aritmetiche sono vincolati a risiedere nei **registri**.

- **Registri**: Locazioni di memoria interne al processore, estremamente veloci (accesso in un solo ciclo di clock).
- **Principio di Progettazione**: "Minori sono le dimensioni, maggiore è la velocità". Limitare il numero di registri (es. 32 nel RISC-V) permette ai segnali elettrici di percorrere distanze minori, mantenendo alte le frequenze di clock.

6.4 Controllo del Flusso e Salti

Per implementare cicli (`while`, `for`) e selezioni (`if-else`), l'Assembly utilizza istruzioni di **salto condizionato**. L'esecuzione di queste istruzioni dipende dal confronto tra valori contenuti nei registri o dallo stato di particolari **flag** (segnali logici) impostati da operazioni precedenti.

6.5 Principi di Progettazione dell'ISA

Nello sviluppo di linguaggi Assembly moderni come il RISC-V, si seguono tre principi fondamentali:

1. **La semplicità favorisce la regolarità:** Istruzioni uniformi semplificano l'hardware.
2. **Minori dimensioni, maggiore velocità:** Pochi registri rendono la CPU più rapida.
3. **Un buon progetto richiede buoni compromessi:** Ad esempio, l'uso di una lunghezza fissa per le istruzioni a scapito della densità del codice.

7 L'Architettura RISC-V

7.1 Caratteristiche Generali

Il RISC-V è un'architettura di tipo **RISC** (Reduced Instruction Set Computer) moderna e open-source, sviluppata all'Università di Berkeley nel 2010. Segue i principi di semplicità e regolarità per favorire l'efficienza dell'hardware e la velocità di esecuzione.

7.2 I Registri nel RISC-V

A differenza della memoria RAM, i registri sono locazioni interne al processore estremamente rapide, il cui accesso avviene tipicamente in un solo ciclo di clock.

- **Quantità:** Il RISC-V dispone di **32 registri** general-purpose, numerati da **x0** a **x31**.
- **Registro x0:** Ha la particolarità di essere cablato al valore costante **0**. Ogni tentativo di scrivervi viene ignorato, il che semplifica molte operazioni comuni.
- **Dimensione:** In base all'implementazione (RV32I o RV64I), i registri possono essere a 32 o 64 bit.

7.3 Organizzazione della Memoria

La memoria è vista come un'ampia sequenza di bit organizzati in gruppi di 8 (**byte**), dove ciascun byte è associato a un indirizzo lineare progressivo.

- **Word e Double Word:** Nonostante l'indirizzamento al byte, il RISC-V opera solitamente su parole (*word*, 4 byte) o parole doppie (*double word*, 8 byte).
- **Endianness:** Il RISC-V utilizza la convenzione **Little Endian**, in cui il byte meno significativo di una parola viene memorizzato all'indirizzo di memoria più basso.
- **Allineamento:** È possibile accedere a parole poste a indirizzi multipli dell'ampiezza della parola stessa (vincolo di allineamento), sebbene il RISC-V sia più flessibile di altre architetture in merito.

7.4 Trasferimento Dati: Load e Store

Le operazioni aritmetico-logiche in RISC-V avvengono esclusivamente tra registri. Per manipolare dati in memoria occorrono istruzioni di trasferimento:

- **Load (ld, lw, lb):** Preleva un dato dalla memoria e lo carica in un registro.
- **Store (sd, sw, sb):** Salva il contenuto di un registro in memoria.
- **Indirizzamento:** L'indirizzo di memoria viene calcolato sommando il contenuto di un **registro base** a uno **spiazzamento** (offset) costante. Esempio: `ld x9, 64(x22)` carica in `x9` la doppia parola all'indirizzo contenuto in `x22` aumentato di 64.

7.5 Rappresentazione delle Istruzioni

Ogni istruzione RISC-V è codificata come un numero binario di **32 bit**. La struttura dell'istruzione è suddivisa in campi specifici che ne definiscono il comportamento:

1. **codop (opcode):** Codice operativo che identifica il tipo di istruzione.
2. **rs1 e rs2:** Registri sorgente (5 bit ciascuno, per indirizzare i 32 registri).
3. **rd:** Registro destinazione per il risultato.
4. **funz3 e funz7:** Campi aggiuntivi per specificare varianti della stessa operazione (es. somma vs sottrazione).

7.6 Pseudo-istruzioni

Per facilitare la scrittura del codice, l'Assembler supporta le **pseudo-istruzioni**, codici mnemonici che non esistono nel linguaggio macchina ma vengono tradotti in istruzioni reali. Ad esempio, `mv x10, x11` (copia registro) viene tradotta internamente come `addi x10, x11, 0`.

8 Tutorato Assembly RISC-V

8.1 Concetti Teorici Fondamentali

- **Registri RISC-V:** L'architettura base a 64 bit dispone di **32 registri** general-purpose da 64 bit (ovvero *double word*).
- **Registro x0:** È un registro speciale cablato al valore costante **0**. Ogni tentativo di modifica viene ignorato; si rivela estremamente utile per semplificare diverse operazioni (es. copia di registri o salti incondizionati).
- **Formati delle Istruzioni:** Il **Formato I** (Immediato) è impiegato per operazioni che richiedono un operando costante (come l'istruzione `addi`) e per gli accessi in memoria (load), permettendo di codificare la costante direttamente nei 32 bit dell'istruzione.
- **Endianness:** Il processore RISC-V utilizza la convenzione **Little Endian**, memorizzando il byte meno significativo all'indirizzo di memoria più basso.

8.2 Traduzione di Costrutti: dal C all'Assembly

Operazioni Aritmetiche, Logiche e Shift

- *Espressioni composte* (es. $a = (b + c) - d$): Si utilizzano registri temporanei per memorizzare i calcoli intermedi.

```
add x5, x11, x12  % x5 = b + c (risultato intermedio)
sub x10, x5, x13  % a = x5 - d
```

- *Moltiplicazioni per potenze di 2* (es. $a = b * 8$): Si ottimizza il calcolo utilizzando l'istruzione di **shift logico a sinistra** (slli). Moltiplicare per 8 (2^3) equivale a traslare i bit di 3 posizioni.

```
slli x10, x11, 3  % a = b << 3 (equivale ad a = b * 8)
```

Accesso alla Memoria e Array

- *Calcolo dell'offset*: L'indirizzamento in RISC-V è al byte. Lavorando con array di *double word* (8 byte), l'indice deve essere sempre moltiplicato per 8 per ricavare lo spiazzamento corretto.
- *Esempio di traduzione* ($A[4] = h + A[2]$):

```
ld x9, 16(x22)    % Carica A[2] (indice 2 * 8 = offset 16)
add x9, x21, x9    % Somma h con A[2]
sd x9, 32(x22)    % Salva il risultato in A[4] (indice 4 * 8 = offset 32)
```

Costrutti Condizionali (if-else)

- *La regola della condizione opposta*: Per tradurre i costrutti di selezione, la prassi comune è valutare la **condizione opposta** rispetto a quella del codice ad alto livello, in modo da poter saltare l'esecuzione del blocco if qualora la condizione sia falsa.
- *Esempio if-else* (if ($i < j$) $f = g + h$; else $f = g - h$);

```
bge x22, x23, ELSE  % Condizione opposta: se i >= j salta a ELSE
add x19, x20, x21    % Ramo IF: f = g + h
beq x0, x0, ESCI     % Salto incondizionato alla fine
ELSE:
sub x19, x20, x21    % Ramo ELSE: f = g - h
ESCI:                % Fine blocco
```

Costrutti Iterativi (while)

- *Gestione dei cicli su Array* (while ($salva[i] == k$) $i += 1$); I cicli richiedono il calcolo dinamico dell'indirizzo base ad ogni singola iterazione, seguito da un controllo sulla condizione di uscita.

```

CICLO:
slli x10, x22, 3    % Moltiplica indice i per 8 per l'offset in byte
add x10, x10, x25    % Aggiunge l'offset all'indirizzo base dell'array
ld x9, 0(x10)        % Carica fisicamente salva[i] dalla memoria
bne x9, x24, ESCI    % Condizione d'uscita: se salva[i] != k, interrompi
addi x22, x22, 1     % Incrementa l'indice i di 1
beq x0, x0, CICLO    % Salto incondizionato per ripetere
ESCI:

```

9 Soluzioni Mockup Complete

1. **Domanda:** Il record di attivazione in RISC-V contiene diverse informazioni quando viene chiamata una funzione, questo include:

- (A) Registri a scorrimento salvati, Registri del tipo “callee-saved” salvati, Indirizzo di ritorno, Registri non a scorrimento salvati, Registri del tipo “caller-saved” salvati, ed altre informazioni.
- (B) Registri a scorrimento salvati, Registri del tipo “caller-saved” salvati, Indirizzo di ritorno, ed altre informazioni.
- (C) Registri non a scorrimento salvati, Registri del tipo “callee-saved” salvati, ed altre informazioni.
- (D) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (A)

Perché è corretta: Nelle convenzioni di chiamata del RISC-V, il record di attivazione (Stack Frame) è la porzione di stack dedicata a una procedura per memorizzare tutte le informazioni necessarie al suo funzionamento. L'opzione A è l'unica che fornisce una lista completa, includendo sia i registri che devono essere preservati dal chiamato (*callee-saved*), sia quelli preservati dal chiamante (*caller-saved*), sia il fondamentale indirizzo di ritorno (*ra*).

Perché scartare le altre: L'opzione (B) omette i registri *callee-saved*, che sono cruciali. L'opzione (C) omette l'indirizzo di ritorno e i *caller-saved*. L'opzione (D) è errata poiché la (A) è esaustiva.

2. **Domanda:** Cosa contiene il registro x5 dopo le seguenti istruzioni?

```

addi x5, x0, 10
addi x6, x0, 21
sll x5, x5, x6

```

- (A) 0000 0000 0001 0100 0000 0000 0000 0000 0000
- (B) 0000 0000 0000 0000 0000 0000 0000 0000 0000
- (C) 0000 0000 0000 0000 0000 0000 0000 0000 1010
- (D) 0000 0000 0000 0001 0100 0000 0000 0000 0000
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (A)

Perché è corretta: Il registro `x5` viene inizializzato a 10 (in binario 1010). L'istruzione `sll` (Shift Left Logical) scorre i bit di `x5` verso sinistra del valore contenuto in `x6` (21 posizioni), riempiendo i bit meno significativi con zeri. Questo equivale a calcolare 10×2^{21} . Spostando la sequenza 1010 di 21 posizioni, otteniamo 0001 0100 seguito da esattamente 20 zeri (cinque blocchi da 4 zeri).

Perché scartare le altre: La (B) presuppone erroneamente che il risultato sia zero. La (C) mostra il valore iniziale di `x5` (10) senza alcuno shift. La (D) ha un numero errato di zeri finali (ne ha solo 16, corrispondenti a uno shift di 17 posizioni anziché 21).

3. **Domanda:** Secondo le convenzioni di chiamata del RISC-V illustrate a lezione, quali registri vengono comunemente utilizzati per il passaggio dei parametri in ingresso alle procedure e per i valori di ritorno?

- (A) Da `x1` a `x8`
- (B) Da `x10` a `x17`
- (C) Da `x18` a `x27`
- (D) Da `x28` a `x31`
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (B)

Perché è corretta: L'interfaccia binaria (*ABI*) del RISC-V dedica in modo standard i registri da `x10` a `x17` (noti come `a0-a7`) al passaggio dei parametri in ingresso. I primi due (`x10` e `x11` / `a0` e `a1`) sono utilizzati anche per restituire i valori di ritorno.

Perché scartare le altre: L'intervallo (A) include registri con scopi diversi (es. `x1` è l'indirizzo di ritorno `ra`, `x2` è lo stack pointer `sp`). L'intervallo (C) racchiude i registri salvati (`s2-s11`). L'intervallo (D) racchiude i registri temporanei (`t3-t6`).

4. **Domanda:** In merito all'ottimizzazione del codice RISC-V, in cosa consiste l'ottimizzazione per il compilatore nota come "ricorsione di coda" (tail recursion)?

- (A) Nel salvataggio sistematico di tutti i registri nello stack prima di effettuare qualsiasi chiamata ricorsiva.
- (B) Nell'uso esclusivo dei registri callee-saved per evitare conflitti con la funzione chiamante.
- (C) Nella sostituzione di un'istruzione `jal` seguita da un ritorno, con un salto incondizionato alla procedura, semplificando la gestione dello stack.
- (D) Nell'esecuzione parallela di più rami della ricorsione utilizzando le pipeline della CPU.
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (C)

Perché è corretta: La *tail recursion* si verifica quando la chiamata ricorsiva è l'ultima istruzione eseguita da una funzione. Il compilatore la ottimizza evitando di creare un nuovo record di attivazione: sostituisce la chiamata a funzione (`jal`) con un semplice salto al prologo (`j`), riutilizzando lo stack frame attuale e prevenendo

lo *stack overflow*.

Perché scartare le altre: La (A) descrive l'esatto opposto (spreco di stack). La (B) riguarda una strategia di allocazione dei registri, non l'ottimizzazione ricorsiva. La (D) descrive parallelismo a livello hardware (ILP), slegato dal concetto logico di tail recursion.

5. **Domanda:** Riporta quale di queste istruzioni è corretta per completare la traduzione del frammento (sostituendo X1): `while (z < g) { f = f + j; z = z + 1; }`

- (A) `add x11, x11, x12`
- (B) `add x12, x12, x11`
- (C) `addi x12, x12, 1`
- (D) `add x13, x12, x11`
- (E) Nessuna delle risposte precedenti è corretta.

Risposta corretta: (B)

Perché è corretta: Nel C, l'operazione principale del ciclo è `f = f + j`. Secondo l'ABI passata come argomento, `f` (terzo parametro) si trova nel registro `x12` (`a2`) e `j` (secondo parametro) in `x11` (`a1`). L'istruzione per sommarli salvando il risultato in `f` è `add x12, x12, x11`.

Perché scartare le altre: La (A) salva il risultato in `j` anziché in `f`. La (C) aggiunge 1 a `f`, ignorando la variabile `j`. La (D) sovrascrive `g` (`x13`) distruggendo la condizione del ciclo.

6. **Domanda:** Quale delle seguenti istruzioni deve sostituire X1 affinché il calcolo dell'indirizzo di memoria dell'array sia corretto? (`long long int v[]`)

- (A) `slli x5, x11, 2`
- (B) `add x5, x11, x0`
- (C) `slli x5, x10, 3`
- (D) `slli x5, x11, 3`

Risposta corretta: (D)

Perché è corretta: Il vettore è di tipo `long long int`, i cui elementi occupano 8 byte ciascuno. Per trovare l'indirizzo dell'elemento *i*-esimo (in `x11`), l'indice deve essere moltiplicato per 8 (il *peso* in byte). Lo shift logico a sinistra di 3 posizioni (`slli ... 3`) equivale a moltiplicare l'indice per $2^3 = 8$.

Perché scartare le altre: La (A) moltiplica per 4, il che sarebbe corretto solo per un normale `int` a 32 bit. La (B) non moltiplica l'indice, assumendo erroneamente un array di `char` (1 byte). La (C) moltiplica l'indirizzo base (`x10`) invece dell'indice, distruggendo del tutto il puntatore alla memoria.

7. **Domanda:** Qual è lo scopo principale dell'istruzione `addi sp, sp, 32` trovata in un epilogo?

- (A) Allocare 32 byte di memoria nello stack per salvare i registri locali in vista di un'ulteriore chiamata.

- (B) Deallocare lo spazio precedentemente riservato nello stack (Record di Attivazione) prima di ritornare al chiamante.
- (C) Caricare i valori di ritorno nel registro `sp`.
- (D) Passare 32 byte di parametri dal callee al caller.

Risposta corretta: (B)

Perché è corretta: In RISC-V lo stack cresce verso indirizzi di memoria *decrementi*. Quando una funzione inizia (prologo), sottrae spazio da `sp` per fare posto. Al termine (epilogo), deve ripristinare il puntatore originale sommando esattamente lo stesso valore (+32), "deallocando" logicamente lo spazio per le future chiamate.

Perché scartare le altre: La (A) confonde l'epilogo con il prologo (l'allocazione si fa con un valore negativo). La (C) è errata perché i valori di ritorno vanno in `a0/a1`, non nello Stack Pointer. La (D) è priva di senso a livello architetturale.

8. **Domanda:** A quale istruzione base corrisponde la pseudo-istruzione `ret`?

- (A) `jal x0, ra`
- (B) `jalr x0, 0(x1)`
- (C) `jr x1`
- (D) `beq x0, x0, x1`

Risposta corretta: (B)

Perché è corretta: La pseudo-istruzione `ret` serve a saltare di ritorno all'indirizzo memorizzato dal chiamante. Il registro standard che contiene questo indirizzo è `x1` (`ra`). A livello hardware, l'istruzione `jalr x0, 0(x1)` salta incondizionatamente all'indirizzo in `x1` (con offset 0) e salva l'istruzione successiva in `x0` (scartandola, poiché `x0` è cablato a 0).

Perché scartare le altre: La (A) usa `jal`, che si basa su un offset relativo al Program Counter e non può saltare a un indirizzo contenuto in un registro. La (C) è a sua volta un'altra pseudo-istruzione (alias di `jalr`). La (D) è un'istruzione di *branch* (salto condizionato), inadatta a salti assoluti basati su registri.

9. **Domanda:** Lavorando con un'architettura RISC-V a 64 bit (RV64), qual è la differenza fondamentale tra `lw` e `lwu`?

- (A) `lw` carica 64 bit dalla memoria, mentre `lwu` ne carica solo 32.
- (B) Entrambe caricano 32 bit dalla memoria, ma `lw` estende il segno sui restanti 32 bit del registro a 64 bit, mentre `lwu` riempie i restanti 32 bit con zeri.
- (C) `lw` carica un dato dalla memoria e lo allinea a sinistra nel registro, `lwu` lo allinea a destra.
- (D) Non esiste alcuna differenza a livello hardware, l'assemblatore sceglie quale usare in base al tipo di dato.

Risposta corretta: (B)

Perché è corretta: Una "word" in RISC-V è di 32 bit. Poiché i registri di RV64 sono a 64 bit, quando si caricano 32 bit bisogna decidere cosa fare della metà superiore del registro. `lw` preserva il valore matematico dei numeri negativi copiando il bit di segno in tutte le posizioni superiori (*sign extension*). `lwu` (Unsigned) tratta

il dato come positivo, riempiendo la parte alta semplicemente con zeri (*zero extension*).

Perché scartare le altre: La (A) è errata perché il caricamento a 64 bit si fa con `ld` (Load Doubleword). La (C) è falsa (tutti i valori sono allineati al Least Significant Bit a destra). La (D) è falsa perché `lw` e `lwu` corrispondono a *opcode* binari hardware fisicamente distinti.

10. **Domanda:** Nelle istruzioni di salto condizionato (come `beq`), come viene calcolato a livello hardware l'indirizzo di memoria dell'istruzione a cui saltare?

- (A) Tramite un indirizzamento assoluto: l'indirizzo a 32/64 bit viene caricato direttamente da un registro base.
- (B) L'indirizzo target viene letto dallo Stack Pointer (`sp`).
- (C) Tramite indirizzamento PC-relative: l'indirizzo target si ottiene sommando il valore attuale del Program Counter (PC) con un offset codificato direttamente all'interno dell'istruzione di salto.
- (D) Il target viene calcolato moltiplicando per 4 il valore contenuto nel registro `ra`.

Risposta corretta: (C)

Perché è corretta: I salti condizionati vengono generalmente utilizzati per saltare a blocchi di codice vicini (`if`, `loop`). RISC-V ottimizza questa operazione utilizzando l'indirizzamento *PC-relative*: calcola la distanza (offset) tra l'istruzione corrente e la destinazione e la somma al Program Counter (PC). Questo permette di risparmiare spazio, poiché un piccolo offset entra comodamente nei 32 bit dell'istruzione.

Perché scartare le altre: La (A) descrive il meccanismo di `jalr`, usato tipicamente per i ritorni a funzione o chiamate tramite puntatore, non per i branch. La (B) coinvolge lo stack, che contiene i dati locali e non le istruzioni da eseguire. La (D) non ha senso architetturale.

11. **Domanda:** Quale istruzione deve sostituire `X1` per permettere l'avanzamento logico corretto del ciclo in `sumV`? (`*v = *u + 1;`)

- (A) `lw a5, 0(a0)`
- (B) `lw a1, 0(a6)`
- (C) `lw a5, 4(a0)`
- (D) `lw a5, -4(a0)`

Risposta corretta: (A)

Perché è corretta: L'istruzione in C `*v = *u + 1` impone prima di tutto di leggere in memoria il valore puntato da `u` per portarlo nel processore. Sapendo che il puntatore `u` è mappato sul registro `a0`, l'istruzione corretta per dereferenziarlo (senza alcuno spiazamento, dato che il puntatore indica esattamente l'elemento attuale) è `lw a5, 0(a0)`, che carica il valore nel registro di appoggio `a5`.

Perché scartare le altre: La (B) usa i registri completamente sbagliati. La (C) usa un offset positivo di 4, saltando inavvertitamente il primo elemento dell'array prima ancora di leggerlo. La (D) usa un offset negativo, leggendo fuori dai limiti previsti dell'array.

12. **Domanda:** Quale combinazione deve sostituire X1, X2, X3 per tradurre la copia di char e il break su '\0'?

- (A) X1: lw x7, 0(x6) | X2: sw x7, 0(x28) | X3: beq x7, x0, end
- (B) X1: lbu x7, 0(x6) | X2: sb x7, 0(x28) | X3: bne x7, x0, end
- (C) X1: lbu x7, 0(x6) | X2: sb x7, 0(x28) | X3: beq x7, x0, end
- (D) X1: lbu x6, 0(x7) | X2: sb x28, 0(x7) | X3: beq x7, x0, end

Risposta corretta: (C)

Perché è corretta: Trattandosi di stringhe, i dati sono di tipo char (1 byte). Occorre caricare un singolo byte senza segno (lbu), salvarlo in destinazione (sb) e infine verificare la condizione di uscita. Il C dice if (c == '\0') break;, quindi dobbiamo uscire dal ciclo (saltare a end) se il carattere in x7 è uguale a zero. L'istruzione per farlo è beq x7, x0, end.

Perché scartare le altre: La (A) usa lw e sw, che spostano blocchi di 4 byte (parole intere) distruggendo la memoria adiacente alla stringa. La (B) esce dal ciclo se il carattere non è zero (bne), interrompendo la copia al primo carattere valido. La (D) inverte i registri per i puntatori, sovrascrivendo gli indirizzi di memoria.

13. **Domanda:** Calcola la rappresentazione di -54 in base 10 in complemento a 2 su 8 bit.

- (A) 1101.1100
- (B) 1100 1010
- (C) 1101 1010
- (D) 0011 0110
- (E) Nessuna delle risposte

Risposta corretta: (B)

Perché è corretta: Si parte dal numero positivo 54_{10} , che in binario su 8 bit è 0011 0110 ($32 + 16 + 4 + 2$). Per ottenere il complemento a 2, si fa prima il complemento a 1 (si invertono tutti i bit: 1100 1001) e poi si somma 1. Aggiungendo 1, l'ultimo bit diventa 0 con riporto, ottenendo 1100 1010.

Perché scartare le altre: La (A) è formattata come numero a virgola fissa. La (C) presenta un errore di calcolo nel bit di peso 16. La (D) è la rappresentazione del numero positivo +54.

14. **Domanda:** Seleziona la rappresentazione binaria del numero 173.

- (A) 1010 1101
- (B) 1011 1101
- (C) 1010 1001
- (D) 1110 1101
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Il metodo più veloce è la somma dei pesi (potenze di 2): $173 = 128 + 32 + 8 + 4 + 1$. Mettendo a 1 le posizioni corrispondenti a questi pesi e a 0 le altre (64, 16, 2), si ottiene esattamente la sequenza 1010 1101.

Perché scartare le altre: La (B) vale 189 (ha il bit del 16 acceso). La (C) vale 169 (manca il bit del 4). La (D) vale 237 (ha il bit del 64 acceso).

15. **Domanda:** Convertire il numero 13.625 in base 10 in binario con rappresentazione a virgola fissa.

- (A) 1101.1100
- (B) 1110.1010
- (C) 1101.1010
- (D) 1101.0101
- (E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: Si scompone in parte intera e frazionaria. La parte intera 13 è 1101 ($8 + 4 + 1$). La parte frazionaria 0.625 può essere vista come $\frac{1}{2} + \frac{1}{8}$, ovvero $0.5 + 0.125$. Nelle potenze di 2 post-virgola ($2^{-1}, 2^{-2}, 2^{-3}$), questo si traduce in 1 sul peso del mezzo, 0 sul quarto, e 1 sull'ottavo: .101. Unendo e aggiungendo uno zero di *padding* a destra (che non altera il valore) otteniamo 1101.1010.

Perché scartare le altre: La (A) equivale a 13.75 ($\frac{1}{2} + \frac{1}{4}$). La (B) calcola male la parte intera (14). La (D) equivale a 13.3125 ($\frac{1}{4} + \frac{1}{16}$).

16. **Domanda:** Quale di queste affermazioni è corretta nella comparazione tra architetture CISC e RISC?

- (A) Entrambe consentono di avere operandi sia in memoria che nei registri.
- (B) Entrambe codificano le istruzioni con un numero di bit costante.
- (C) L'architettura RISC non supporta operandi "immediati", mentre CISC li supporta.
- (D) L'architettura RISC non supporta l'esecuzione di istruzioni diverse in base al valore di verità di una condizione.
- (E) Entrambe dispongono di operazioni che consentono di scambiare dati tra la memoria e i registri.

Risposta corretta: (E)

Perché è corretta: Che un'architettura sia basata su istruzioni semplici (RISC) o complesse (CISC), lo scambio di dati tra la memoria centrale e la CPU è un'esigenza fondamentale. Entrambe possiedono operazioni dedicate a questo scopo (come `load/store` nel RISC-V o `MOV` nell'x86).

Perché scartare le altre: La (A) è falsa perché il RISC forza l'uso esclusivo di registri per le operazioni ALU, vietando operandi in memoria. La (B) è falsa perché il CISC (come l'Intel x86) usa istruzioni a lunghezza variabile. La (C) è falsa (il RISC usa moltissimo gli immediati, es. `addi`). La (D) è falsa (il RISC ha eccome i salti condizionati, es. `beq`).

17. **Domanda:** Il registro `x5` contiene 11. Quale sarà il valore di `x6` dopo `slli x6, x5, 2` seguito da `xori x6, x6, 15`?

- (A) `x6 = 0000 0000 0000 0000 0000 0000 0010 0011`
- (B) `x6 = 0000 0000 0000 0000 0000 0000 0010 1100`
- (C) `x6 = 0000 0000 0000 0000 0000 0000 0011 0011`
- (D) `x6 = 0000 0000 0000 0000 0000 0000 0010 0111`
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Il valore decimale 11 è `0000 1011`. L'istruzione `slli` lo trasla a sinistra di 2 bit, diventando `0010 1100` (equivalente a moltiplicare per 4, cioè 44). L'istruzione `xori` fa un XOR bit a bit con 15 (`0000 1111`). La regola dell'XOR dice che l'uscita è 1 se e solo se i bit in ingresso sono diversi. Operando `0010 1100 XOR 0000 1111`, otteniamo esattamente `0010 0011` (il valore 35).

Perché scartare le altre: La (B) ignora completamente il passaggio finale dell'XOR, fermandosi al primo shift. La (C) è il risultato di un'operazione OR invece di XOR. La (D) è il risultato di una banale addizione (+15) invece dell'XOR logico.

18. **Domanda:** Quale istruzione va inserita al posto di `X1` nel ciclo `while(arr[i] != 0)`?

- (A) `bne a4, zero, .L3`
- (B) `beq a4, zero, .L2`
- (C) `bne a0, zero, .L3`
- (D) `beq a5, zero, .L2`
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: `X1` si trova alla fine del corpo del ciclo, fungendo da condizione per saltare indietro (`.L3`) e ripetere l'iterazione. L'elemento corrente dell'array è appena stato caricato nel registro `a4`. Dato che il C richiede di ciclare finché l'elemento è diverso da zero, l'istruzione deve ordinare "salta a `.L3` se `a4` NON È UGUALE a zero", ovvero `bne a4, zero, .L3`.

Perché scartare le altre: La (B) salta alla fine del ciclo se il valore è zero; sebbene logicamente corretto per uscire, posizionata in `X1` richiederebbe poi un salto incondizionato esplicito per tornare su, allungando il codice inutilmente. La (C) controlla il contatore `a0` invece dell'elemento letto dalla memoria. La (D) controlla l'indirizzo del puntatore `a5`, non il dato che esso contiene.

19. **Domanda:** Effettua la seguente sottrazione in binario: `1010 1100 - 0101 0111`.

- (A) `0101 0101`
- (B) `0101 1101`
- (C) `0110 0101`
- (D) `0100 0101`
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Eseguendo la sottrazione colonna per colonna da destra verso sinistra e gestendo i prestiti (borrow), si ottiene 0101 0101. Un metodo di riprova infallibile è convertire i numeri in base 10: $172 - 87 = 85$. Convertendo 85 in binario ($64 + 16 + 4 + 1$) otteniamo proprio 0101 0101.

Perché scartare le altre: Le altre opzioni presentano semplici errori aritmetici nella propagazione dei prestiti tra i bit adiacenti.

20. **Domanda:** Rappresenta il numero decimale -5.75 nella codifica dello standard IEEE-754 a singola precisione (32 bit).

- (A) 0100 0000 1011 1000 0000 0000 0000 0000
- (B) 1100 0000 1101 1000 0000 0000 0000 0000
- (C) 1100 0000 1011 1000 0000 0000 0000 0000
- (D) 1100 0001 0011 1000 0000 0000 0000 0000
- (E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: 1) Segno: negativo $\rightarrow$ 1. 2) Conversione di 5.75 in binario: 101.11. 3) Normalizzazione: spostiamo la virgola di 2 posizioni $\rightarrow 1.0111 \times 2^2$. 4) Esponente con Bias: l'esponente base è 2, sum bias (127) = 129. 129 in binario è 1000 0001. 5) Mantissa: prendiamo 0111 (rimuovendo l'1 implicito) e riempiamo con zeri fino a 23 bit. Mettendo in fila i campi [1] [10000001] [01110000...] e raggruppando a blocchi di 4, otteniamo esattamente l'opzione C.

Perché scartare le altre: La (A) imposta il bit di segno a 0 (come fosse positivo). La (B) riporta una mantissa sbagliata (presume una frazione diversa da .75). La (D) ha un esponente pari a 130 anziché 129 (corrisponderebbe al numero -11.5).

21. **Domanda:** Converti il numero decimale positivo 213 nella sua rappresentazione binaria.

- (A) 1101 0101
- (B) 1100 0101
- (C) 1101 0011
- (D) 1110 0101
- (E) Nessuna delle risposte

Risposta corretta: (A)

Perché è corretta: Utilizzando il metodo delle potenze decrescenti di due: $213 - 128 = 85$. $85 - 64 = 21$. $21 - 16 = 5$. $5 - 4 = 1$. $1 - 1 = 0$. Le potenze utilizzate sono 128, 64, 16, 4, 1. Accendendo i bit corrispondenti (1) e tenendo spenti gli altri (0), la sequenza prodotta è 1101 0101.

Perché scartare le altre: Ciascuna opzione errata somma un totale differente: la (B) equivale a 197, la (C) a 211 e la (D) a 229.

22. **Domanda:** Quale istruzione va inserita al posto di X1 nel calcolo del $\max(a, b)$?

- (A) blt a0, a1, .L2

- (B) `bgt a0, a1, .L2`
- (C) `bge a0, a1, .L2`
- (D) `bne a0, a1, .L2`
- (E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: La logica Assembly assume temporaneamente che `a` (in `a0`) sia il massimo copiandolo in `a5`. L'istruzione `X1` decide se dobbiamo scavalcare (saltare) il ramo dell'*else* (che eleggerebbe `b` come massimo). Dato che il C dice `if (a > b)`, dobbiamo saltare l'*else* se e solo se la condizione `a >= b` è vera (che copre sia `a > b` sia `a == b`, per cui è indifferente chi dei due restituiamo). Pertanto l'istruzione per "saltare a `.L2` se `a0 >= a1`" è `bge a0, a1, .L2`.

Perché scartare le altre: La (A) salterebbe l'*else* se `a < b`, producendo il risultato opposto (la funzione restituirebbe il minimo). La (B) salta se `a > b`, ma fallirebbe nel caso in cui siano uguali (eseguendo un passaggio inutile o creando potenziali inefficienze logiche). La (D) salta sulla disuguaglianza pura, un criterio non adatto a un operatore maggiore/minore.

23. **Domanda:** Quale istruzione va inserita al posto di `X1` per avanzare di tre posizioni nel ciclo `somma_ogni_tre`?

- (A) `addi a5, a5, 3`
- (B) `slli a5, a5, 2`
- (C) `addi a5, a5, 12`
- (D) `addi a4, a4, 12`
- (E) Nessuna delle risposte

Risposta corretta: (C)

Perché è corretta: `X1` si occupa di aggiornare il *puntatore* base dell'array (`a5`) in modo che alla successiva iterazione l'istruzione `lw` legga l'elemento tre celle più avanti. In C, gli array di `int` usano celle di 4 byte ciascuna. Avanzare di 3 posizioni significa spostarsi in avanti di $3 \times 4 = 12$ byte fisici in memoria. L'istruzione corretta somma quindi 12 all'indirizzo in `a5`.

Perché scartare le altre: La (A) somma 3 byte, disallineando la lettura e corrompendo l'accesso alla memoria. La (B) moltiplica per 4 l'intero indirizzo base fisico, distruggendolo del tutto. La (D) altera l'indice del ciclo (`a4`), che invece l'Assembly precedente aggiornava già correttamente di +3 dal punto di vista logico.

24. **Domanda:** Quale istruzione va inserita al posto di `X1` per implementare il costrutto `if(arr[i] > 0)`?

- (A) `bge a4, zero, .L3`
- (B) `ble a4, zero, .L3`
- (C) `bgt a4, zero, .L3`
- (D) `blt a4, zero, .L3`
- (E) Nessuna delle risposte

Risposta corretta: (B)

Perché è corretta: Come di consueto in Assembly, per decidere se eseguire o meno un blocco `if`, si valuta la **condizione opposta** rispetto al codice `C`. L'obiettivo è saltare l'operazione di incremento del contatore (`count++`) e passare alla successiva iterazione (`j .L3`). La condizione opposta a "maggiore di zero" (`> 0`) è "minore o uguale a zero" (`<= 0`). L'istruzione per "salta se `a4` è minore o uguale a zero" è `ble a4, zero, .L3`.

Perché scartare le altre: La (A) e la (C) si basano sulla condizione diretta e non opposta, il che comporterebbe il salto del blocco proprio quando andrebbe eseguito. La (D) salta unicamente se il numero è strettamente negativo, fallendo però nel caso in cui il numero sia esattamente zero (che non va conteggiato e quindi richiederebbe di saltare il blocco).

10 Flusso di Controllo e Procedure

10.1 Istruzioni di Salto Condizionato

Per implementare costrutti di selezione come `if-else` e cicli (`while`, `for`), l'Assembly utilizza istruzioni di salto condizionato che alterano l'indirizzo della prossima istruzione da eseguire solo se una specifica condizione logica è verificata. Nel RISC-V, le istruzioni principali confrontano il contenuto di due registri.

- `beq rs1, rs2, Label`: Effettua il salto all'etichetta specificata se il valore in `rs1` è uguale a quello in `rs2`.
- `bne rs1, rs2, Label`: Effettua il salto se i valori nei due registri sono diversi.
- **Altri confronti**: Esistono istruzioni per verificare se un valore è minore di (`blt`) o maggiore/uguale (`bge`), sia per numeri con segno che senza segno (`bltu`, `bgeu`).

Le sequenze di istruzioni comprese tra due salti condizionati sono denominate **blocchi di base**; l'identificazione di tali blocchi è una delle prime fasi compiute dal compilatore per tradurre i cicli dei linguaggi ad alto livello.

10.2 Protocollo di Chiamata delle Procedure

L'utilità di un calcolatore dipende dalla possibilità di invocare procedure (o funzioni), viste come "scatole nere" che eseguono un compito specifico. Per garantire l'interoperabilità tra diverse parti di un programma, viene definito un protocollo (o convenzione) di chiamata.

1. **Passaggio parametri**: L'idea di base è utilizzare i registri, poiché sono il meccanismo più veloce; il RISC-V dedica i registri da `x10` a `x17` (`a0-a7`) per i parametri in ingresso e per i valori di ritorno.
2. **Salvataggio indirizzo di ritorno**: Il registro `x1` (`ra`, *return address*) memorizza l'indirizzo a cui tornare al termine della procedura.
3. **Istruzioni di salto**: L'istruzione `jal` (*jump and link*) esegue il salto e salva simultaneamente l'indirizzo dell'istruzione successiva in `ra`.
4. **Ritorno dal controllo**: Al termine, la procedura esegue `jalr x0, 0(x1)` per tornare al punto di partenza.

10.3 Gestione dello Stack e Record di Attivazione

Se i registri disponibili non sono sufficienti per contenere tutti i parametri o le variabili locali, si ricorre allo **Stack** (pila).

- **Stack Pointer (sp)**: Il registro **x2** punta alla cima dello stack, che per convenzione cresce verso indirizzi di memoria decrescenti.
- **Prologo**: All'inizio di una procedura, il chiamante decrementa **sp** per allocare lo spazio necessario e salva i registri che devono essere preservati (operazione di *push*).
- **Epilogo**: Prima del ritorno, si ripristinano i valori originali dei registri caricandoli dallo stack (operazione di *pop*) e si incrementa **sp** per liberare la memoria.
- **Frame Pointer (fp)**: Alcuni programmi usano il registro **x8** (**fp**) come base stabile per individuare le variabili locali tramite uno spiazzamento costante, poiché **sp** può variare durante l'esecuzione.

10.4 Funzioni Foglia e Non Foglia

Si definisce **funzione foglia** una procedura che non chiama altre funzioni al suo interno. In ARM queste sono gestite in modo più semplice rispetto al RISC-V, poiché non è sempre obbligatorio salvare l'indirizzo di ritorno se non si effettuano ulteriori chiamate. Al contrario, una **funzione non foglia** deve necessariamente salvare il proprio registro **ra** sullo stack nel prologo, altrimenti verrebbe sovrascritto dalla successiva istruzione **jal**.

10.5 Organizzazione della Memoria

Il calcolatore organizza la memoria in segmenti logici distinti :

- **Stack**: Utilizzato per le variabili locali e i record di attivazione; procede verso indirizzi decrescenti .
- **Heap**: Destinato ai dati dinamici (allocati tramite **malloc** o **new**); procede verso indirizzi crescenti .
- **Dati Statici**: Locazioni per variabili globali, accessibili tramite il registro **x3** (**gp**, *global pointer*).
- **Testo (Code)**: L'area di memoria che contiene le istruzioni macchina del programma in esecuzione .

11 L'Architettura Intel x86

11.1 Evoluzione e Caratteristiche CISC

L'architettura Intel rappresenta lo standard dominante per desktop, laptop e server. A differenza del RISC-V, si tratta di un'architettura di tipo **CISC** (Complex Instruction Set Computer), caratterizzata da un set di istruzioni estremamente vasto e complesso, frutto di un'evoluzione iniziata negli anni '70. Una caratteristica fondamentale è la **retrocompatibilità**: le moderne CPU a 64 bit sono ancora in grado di eseguire codice scritto per i vecchi processori a 8 o 16 bit, il che comporta una notevole complessità hardware.

11.2 I Registri Intel

Il set di registri Intel a 64 bit è composto da 16 registri general-purpose, tutti identificati dal prefisso %.

- **Nomi classici:** %rax, %rbx, %rcx, %rdx, %rsi, %rdi, %rbp, %rsp.
- **Nuovi registri:** Da %r8 a %r15.
- **Struttura gerarchica:** Ogni registro a 64 bit include sottoregistri per la compatibilità con le ampiezze minori. Ad esempio, il registro %rax (64 bit) contiene %eax (32 bit), %ax (16 bit) e %al (8 bit).
- **Registri speciali:** %rsp funge da *stack pointer*, mentre %rbp è il *base pointer* (puntatore al record di attivazione).

11.3 Formato delle Istruzioni e Operandi

A differenza del RISC-V, le istruzioni Intel sono prevalentemente a **2 operandi**, dove il secondo operando funge anche da destinazione implicita (*sorgente, destinazione*).

- **Flessibilità:** Le istruzioni aritmetico-logiche possono operare direttamente sulla memoria.
- **Vincolo memoria-memoria:** Non è consentito avere entrambi gli operandi in memoria in una singola istruzione; è necessario utilizzare un registro di appoggio.
- **Suffissi di ampiezza:** L'ampiezza dei dati è specificata da un carattere finale nell'opcode: b (8 bit), w (16 bit), l (32 bit) e q (64 bit).

11.4 Modalità di Indirizzamento

L'architettura Intel offre una modalità di indirizzamento molto potente chiamata **indiretto scalato**, che permette di calcolare l'indirizzo finale con la formula:

$$\text{Indirizzo} = \text{base} + (\text{indice} \times \text{scala}) + \text{spiazzamento}$$

dove la scala può assumere solo i valori 1, 2, 4 o 8. Questa struttura è ideale per scorrere array di diversi tipi di dati con una singola istruzione.

11.5 Istruzioni Particolari

- **leaq (Load Effective Address):** Utilizza l'hardware del calcolo degli indirizzi per eseguire operazioni aritmetiche (come somme e shift combinati) senza accedere effettivamente alla memoria.
- **inc / dec:** Sommano o sottraggono 1. Sono preferite alla add \$1 perché occupano meno byte nella codifica binaria dell'istruzione.

11.6 Convenzione di Chiamata (ABI)

Per garantire la compatibilità tra programmi e librerie, l'interfaccia binaria (*System V ABI*) stabilisce quanto segue:

- **Passaggio parametri:** I primi 6 argomenti interi o puntatori vengono passati nei registri `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` e `%r9`. Gli argomenti successivi vengono caricati sullo **stack**.
- **Valore di ritorno:** Viene inserito nel registro `%rax`.
- **Registri preservati:** Il chiamato deve salvare e ripristinare `%rbp`, `%rbx` e `%r12-%r15` se intende utilizzarli.

12 L'Architettura Intel x86

12.1 Evoluzione e Caratteristiche CISC

L'architettura Intel rappresenta lo standard dominante per desktop, laptop e server. A differenza del RISC-V, si tratta di un'architettura di tipo **CISC** (Complex Instruction Set Computer), caratterizzata da un set di istruzioni estremamente vasto e complesso, frutto di un'evoluzione iniziata negli anni '70. Una caratteristica fondamentale è la **retrocompatibilità**: le moderne CPU a 64 bit sono ancora in grado di eseguire codice scritto per i vecchi processori a 8 o 16 bit, il che comporta una notevole complessità hardware.

12.2 I Registri Intel

Il set di registri Intel a 64 bit è composto da 16 registri general-purpose, tutti identificati dal prefisso `%`.

- **Nomi classici:** `%rax`, `%rbx`, `%rcx`, `%rdx`, `%rsi`, `%rdi`, `%rbp`, `%rsp`.
- **Nuovi registri:** Da `%r8` a `%r15`.
- **Struttura gerarchica:** Ogni registro a 64 bit include sottoregistri per la compatibilità con le ampiezze minori. Ad esempio, il registro `%rax` (64 bit) contiene `%eax` (32 bit), `%ax` (16 bit) e `%al` (8 bit).
- **Registri speciali:** `%rsp` funge da *stack pointer*, mentre `%rbp` è il *base pointer* (puntatore al record di attivazione).

12.3 Formato delle Istruzioni e Operandi

A differenza del RISC-V, le istruzioni Intel sono prevalentemente a **2 operandi**, dove il secondo operando funge anche da destinazione implicita (*sorgente, destinazione*).

- **Flessibilità:** Le istruzioni aritmetico-logiche possono operare direttamente sulla memoria.
- **Vincolo memoria-memoria:** Non è consentito avere entrambi gli operandi in memoria in una singola istruzione; è necessario utilizzare un registro di appoggio.
- **Suffissi di ampiezza:** L'ampiezza dei dati è specificata da un carattere finale nell'opcode: `b` (8 bit), `w` (16 bit), `l` (32 bit) e `q` (64 bit).

12.4 Modalità di Indirizzamento

L'architettura Intel offre una modalità di indirizzamento molto potente chiamata **indiretto scalato**, che permette di calcolare l'indirizzo finale con la formula:

$$\text{Indirizzo} = \text{base} + (\text{indice} \times \text{scala}) + \text{spiazzamento}$$

dove la scala può assumere solo i valori 1, 2, 4 o 8. Questa struttura è ideale per scorrere array di diversi tipi di dati con una singola istruzione.

12.5 Istruzioni Particolari

- **leaq (Load Effective Address)**: Utilizza l'hardware del calcolo degli indirizzi per eseguire operazioni aritmetiche (come somme e shift combinati) senza accedere effettivamente alla memoria.
- **inc / dec**: Sommano o sottraggono 1. Sono preferite alla **add \$1** perché occupano meno byte nella codifica binaria dell'istruzione.

12.6 Convenzione di Chiamata (ABI)

Per garantire la compatibilità tra programmi e librerie, l'interfaccia binaria (*System V ABI*) stabilisce quanto segue:

- **Passaggio parametri**: I primi 6 argomenti interi o puntatori vengono passati nei registri **%rdi**, **%rsi**, **%rdx**, **%rcx**, **%r8** e **%r9**. Gli argomenti successivi vengono caricati sullo **stack**.
- **Valore di ritorno**: Viene inserito nel registro **%rax**.
- **Registri preservati**: Il chiamato deve salvare e ripristinare **%rbp**, **%rbx** e **%r12-%r15** se intende utilizzarli.

13 L'Architettura ARM

13.1 Origini e Caratteristiche

L'architettura ARM (*Advanced RISC Machine*), nata negli anni '80 come Acorn RISC Machine, è oggi la più diffusa nei sistemi embedded, smartphone e tablet. Si definisce un'architettura RISC "pragmatica" poiché, pur mantenendo i principi di efficienza tipici del RISC, introduce modalità di indirizzamento e istruzioni potenti che ricordano il mondo CISC per ottimizzare la densità del codice e il risparmio energetico.

13.2 I Registri ARM

ARM dispone di **16 registri a 32 bit**, denominati da **r0** a **r15**.

- **r0-r12**: Registri general-purpose. Per convenzione ABI, **r0-r3** sono usati per il passaggio dei parametri e i valori di ritorno.
- **r13 (sp)**: *Stack Pointer*, punta alla cima dello stack.

- **r14 (lr)**: *Link Register*, memorizza l'indirizzo di ritorno dalle subroutine (analogo a **ra** in RISC-V).
- **r15 (pc)**: *Program Counter*. A differenza di molte altre architetture, il PC è accessibile direttamente come registro.
- **CPSR/APSR**: Registri di stato che contengono i flag (**N**egative, **Z**ero, **C**arry, **V**overflow) usati per l'esecuzione condizionale.

13.3 Formato delle Istruzioni e Flessibilità

Le istruzioni ARM sono tipicamente a **3 argomenti** (*destinazione, sorgente1, sorgente2*). Una caratteristica distintiva è che il terzo argomento può essere un valore immediato o un registro sottoposto a un'operazione di **shift o rotazione** (es. **LSL**, **LSR**) integrata nell'istruzione stessa senza costi aggiuntivi in cicli di clock.

13.4 Esecuzione Condizionale

Quasi tutte le istruzioni ARM possono essere eseguite in modo condizionale aggiungendo un suffisso di due lettere al codice mnemonico.

- **eq** (equal), **ne** (not equal), **lt** (less than), **gt** (greater than), ecc.
- Questo approccio permette di evitare molti salti condizionati, migliorando l'efficienza della pipeline.

13.5 Modalità di Indirizzamento e Write-back

ARM offre modalità di indirizzamento alla memoria molto avanzate (tramite **ldr** e **str**):

- **Base + Offset/Indice**: L'indirizzo è calcolato come **[base + offset]**.
- **Pre-indexing**: L'indirizzo viene aggiornato *prima* dell'accesso. Se seguito dal simbolo **!**, il valore aggiornato viene salvato nel registro base (**write-back**).
- **Post-indexing**: L'accesso avviene all'indirizzo contenuto nel registro base, che viene aggiornato *dopo* l'operazione. Questa modalità è estremamente utile per scorrere array in cicli **while** o **for**.

13.6 Istruzioni Multiple (LDM e STM)

ARM permette di caricare o salvare **multipli registri** con una singola istruzione (**ldm** e **stm**). Queste sono fondamentali nel prologo e nell'epilogo delle funzioni per gestire lo stack in modo efficiente, permettendo di salvare l'intero stato dei registri e l'indirizzo di ritorno con una sola riga di codice.

14 La Toolchain: dalla Sorgente all'Esecuibile

14.1 Il Processo di Traduzione

Poiché una CPU è in grado di comprendere ed eseguire esclusivamente il proprio **linguaggio macchina** (sequenze di bit), i programmi scritti in linguaggi ad alto livello o in Assembly devono essere trasformati attraverso una catena di strumenti software chiamata **toolchain**.

- **Compilatore:** Traduce il codice sorgente (es. C) in un file in linguaggio Assembly.
- **Assembler:** Trasforma il file Assembly in un **file oggetto** binario.
- **Linker:** Unisce diversi file oggetto e librerie in un unico **file eseguibile** finale.

14.2 Il Ruolo dell'Assembler

L'Assembler non si limita alla semplice traduzione uno-a-uno dei codici mnemonici in bit. Le sue funzioni principali includono:

- **Gestione delle Pseudo-istruzioni:** Traduce comandi simbolici utili al programmatore in istruzioni reali. Ad esempio, nel RISC-V:
 - `mv x10, x11` viene tradotto in `addi x10, x11, 0`.
 - `li x9, 123` (carica immediato) viene tradotto in `addi x9, x0, 123`.
 - `j Label` (salto incondizionato) viene tradotto in `jal x0, Label`.
- **Gestione delle Label:** Sostituisce i nomi simbolici dei salti con gli indirizzi binari corrispondenti (spesso calcolati come offset relativi al Program Counter).
- **Gestione dei Dati:** Converte i numeri (decimali o esadecimali) e le stringhe di testo in sequenze di bit.

14.3 Struttura del File Oggetto

Il file prodotto dall'Assembler è organizzato in segmenti logici distinti:

1. **Header:** Specifica la dimensione e la posizione degli altri segmenti.
2. **Text Segment:** Contiene le istruzioni in linguaggio macchina.
3. **Data Segment:** Contiene i dati statici (variabili globali) e le costanti.
4. **Tabella dei Simboli:** Elenca le etichette definite nel file che potrebbero essere usate da altri moduli (es. nomi di funzioni).
5. **Tabella di Rilocalizzazione:** Indica quali istruzioni dipendono da indirizzi assoluti che verranno stabiliti solo nella fase finale.

14.4 Il Linker e la Creazione dell'Eseguibile

Il **Linker** ha il compito di "cucire" insieme i vari moduli. Le sue fasi principali sono:

- **Risoluzione dei simboli:** Associa ogni riferimento a una funzione o variabile esterna alla sua effettiva definizione.
- **Calcolo degli indirizzi:** Determina la posizione finale di ogni istruzione e dato nella memoria del calcolatore, aggiornando i riferimenti nel codice.

Il risultato è un file eseguibile pronto per essere caricato in memoria dal **Loader** del Sistema Operativo.

14.5 Linking Statico e Dinamico

- **Linking Statico:** Tutte le librerie necessarie vengono incluse direttamente nel file eseguibile. Questo lo rende più grande ma autonomo.
- **Linking Dinamico:** Il programma contiene solo riferimenti a librerie esterne (es. `.dll` o `.so`). Il collegamento avviene solo quando il programma viene caricato (*non-lazy*) o quando la funzione viene effettivamente chiamata (*lazy linking*), risparmiando spazio in memoria.